```
$ ./whyrunner --tcc-ii --parcial
```

WhyRunner

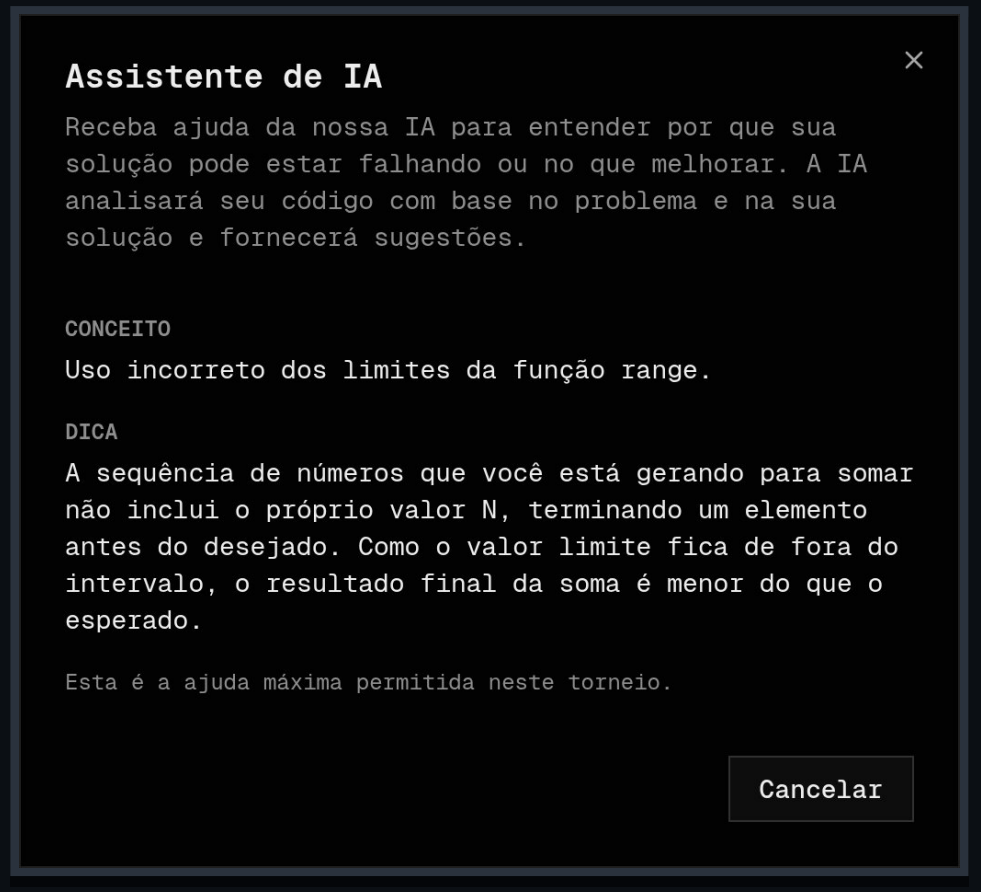
Juiz online para o ensino de programação, com dicas de IA que o professor controla

Juan Israel dos Anjos

Orientadora: Suellen Mapa

Bacharelado em Sistemas de Informação · IFMG Ouro Branco

TCC II · Apresentação parcial · 16/10/2026



Dica da IA em dois níveis, dentro de um torneio

Juízes online corrigem código. Ensinar é outra coisa

Juiz online tradicional

✗ Wrong Answer

Caso de teste 3 de 10 · sem explicação

BOCA, Run Codes: veredito binário, pensado para competição

Motivação: anos de maratona de programação usando esses sistemas como aluno.

Escopo do TCC

FAZ

- Torneios
- Turmas e aulas
- Execução isolada · 6 linguagens
- Dicas de IA graduadas
- Restrições de solução

NÃO FAZ

- Detecção de plágio
- Avaliar estilo de código
- Substituir a avaliação do professor

O aluno recebe "Wrong Answer" e para por aí

Problema

Juízes priorizam justiça e velocidade. O feedback é um veredito binário.

Justificativa

Erro sem explicação não vira aprendizado. E o professor não consegue montar atividades nem exigir uma abordagem.

PERGUNTA DE PESQUISA

Feedback por IA e atividades curadas pelo professor tornam um juiz online mais útil para o ensino?

Objetivo geral: desenvolver e avaliar um juiz online voltado ao ensino de programação.

1. Executar código com segurança em várias linguagens
2. Dar dicas por IA sem entregar a solução
3. Deixar o professor montar aulas e exigir algoritmos
4. Avaliar a usabilidade com alunos e professor

● em andamento

Nenhum dos sistemas analisados ensina, só corrige

Sistema	Feedback IA	Linguagens	Tempo real	Aulas curadas	Restrições	Portugol + blocos
Ataux (2021)	—	Limitadas	—	—	—	—
GOJ – UnB (2022)	—	Moderadas	Limitado	—	—	—
Run Codes	—	Muitas	●	—	—	—
BOCA (2004)	—	Moderadas	●	—	—	—
WhyRunner	●	6	●	●	●	●

Diferencial: dicas por IA, aulas montadas pelo professor, Portugol com blocos e restrições de solução: verificar *como* o aluno resolveu, não só se a saída está certa.

Ciclos iterativos: cada versão corrigiu a anterior

	TENTADO		ADOTADO	POR QUÊ
Fila	~~BullMQ → AWS SQS~~	→	**PostgreSQL LISTEN/NOTIFY**	sem polling ocioso nem outro serviço pago
Backend	~~API NestJS separada~~	→	**Next.js Server Actions**	tipos e autenticação não ficam duplicados
Banco	~~Supabase~~	→	**PostgreSQL + Drizzle**	liberdade para lógica no servidor
Sandbox	~~Docker por submissão~~	→	**isolate (usado na IOI; MAREŠ; BLACKHAM, 2012)**	Docker exigia host privilegiado

Levantamento: literatura + conversas com alunos + experiência própria

MVP testado informalmente no Discord

Dois serviços que só conversam pelo banco



1. Aluno submete em problema, torneio ou aula
2. Server Action grava PENDING e dispara NOTIFY
3. Judge recebe e busca os casos de teste
4. isolate compila e executa com limites
5. Checa restrições: análise estática, depois IA
6. Veredito gravado; aluno notificado

```
Command::new("isolate")
    .arg(format!("--time={}", limits.time_secs))
    .arg(format!("--mem={}", limits.mem_kb))
    .arg(format!("--processes={}", limits.processes))
```

`judge/src/isolate.rs`

Caixa sem rede, com limite de tempo, memória e processos: o sandbox da IOI (MAREŠ; BLACKHAM, 2012).

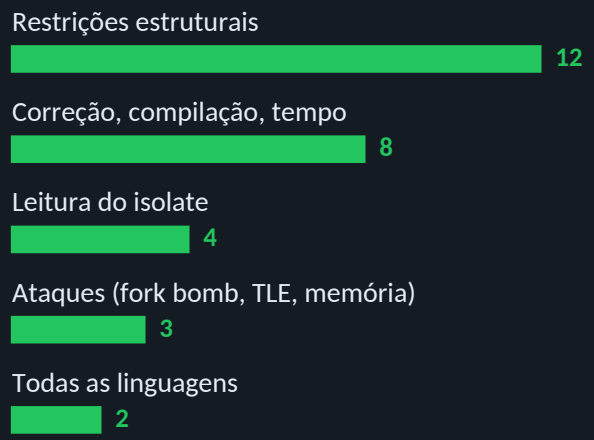
C · C++ · Java · Python · Rust · Portugol

O judge é testado sozinho, sem o site

FEITO

Validação técnica

29 testes automatizados no judge



12 — **Restrições estruturais**
Laços aninhados e construções proibidas em cada linguagem, incluindo casos que não podem disparar (texto em strings).

8 — **Correção, compilação, tempo**
Aceito, resposta errada, erro de compilação com a saída do compilador, limite de tempo do problema.

4 — **Leitura do isolate**
Interpretação do arquivo de metadados: tempo, memória, código de saída e sinal.

3 — **Ataques**
Fork bomb, laço infinito e estouro de memória ficam contidos na caixa.

2 — **Todas as linguagens**
C, C++, Java, Python, Rust e Portugol compilam e passam num eco simples.

Nenhum problema é publicado sem uma solução que passa



Solução de referência gerada pela IA, aprovada contra todos os casos de teste

1. **Escolhe a linguagem**
   Qualquer uma das 6 que o judge executa.

2. **Gera com IA ou cola**
   A IA escreve uma solução de referência a partir da descrição e dos casos salvos.

3. **Valida no judge**
   A solução roda contra todos os casos. Saída diferente = caso de teste errado.

4. **Publicação travada**
   Mudou um teste? É preciso validar de novo antes de publicar.

Três lugares para submeter, três propósitos

PROBLEMA



Para quê
Treino livre no banco de problemas

Quem monta
Autor publica; aluno resolve no seu ritmo

IA
Sem IA: veredito e casos de exemplo

TORNEIO

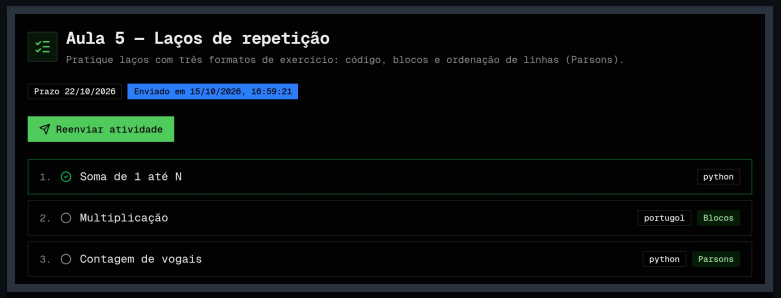


Para quê
Competir ou avaliar com tempo marcado

Quem monta
Professor define problemas, período e acesso

IA
IA no nível que o professor escolher

AULA

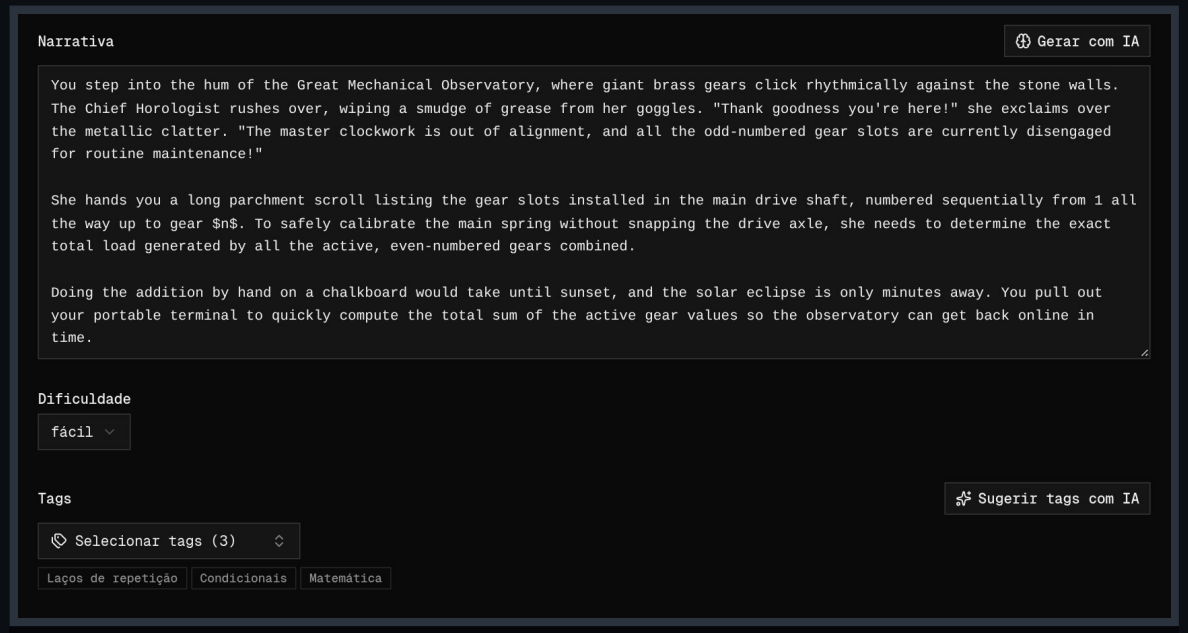


Para quê
Aprender em sequência, com prazo e nota

Quem monta
Professor monta a aula e comenta cada resposta

IA
Dica livre · código, blocos ou Parsons

A IA ajuda o professor a escrever o problema



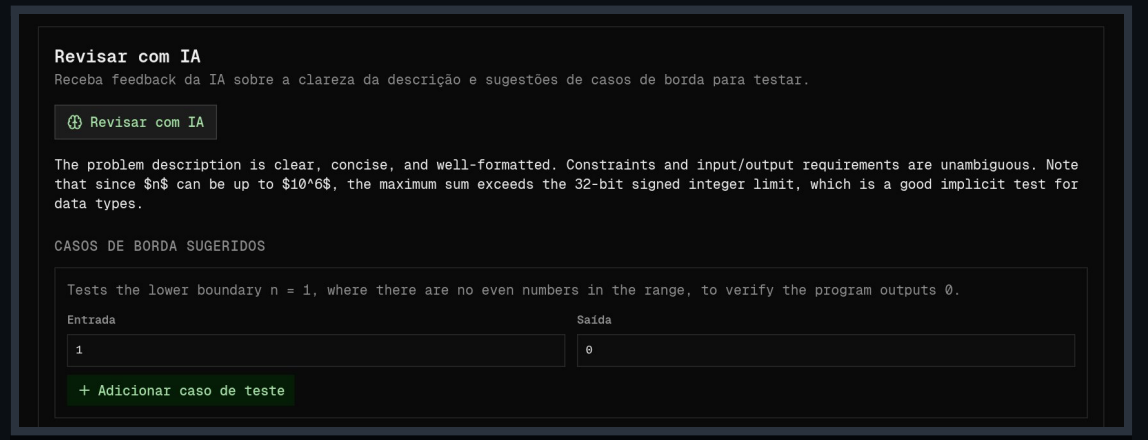
Narrativa e tags geradas pela IA

Narrativa: um clique gera uma história para o enunciado.

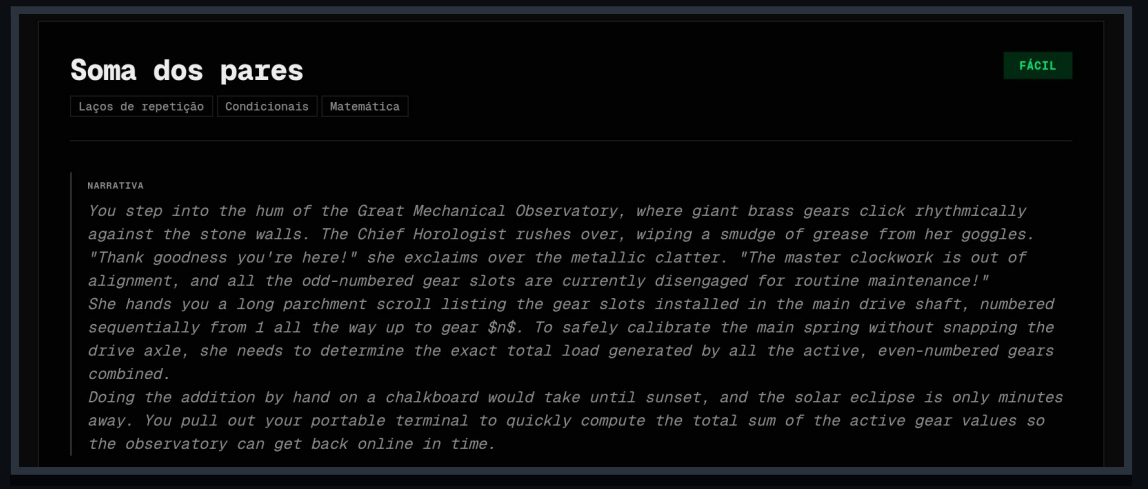
Tags: a IA sugere os temas do problema.

Revisão: aponta ambiguidades e sugere casos de borda.

Pré-visualização: o problema como o aluno vai ver (abaixo, à direita).



Revisão com IA: casos de borda que faltam



Achar o problema certo pelo código, tema ou dificuldade



Busca por código (DEMO) com o filtro de tags aberto

Busca
por título ou pelo código do problema (DEMO-101).

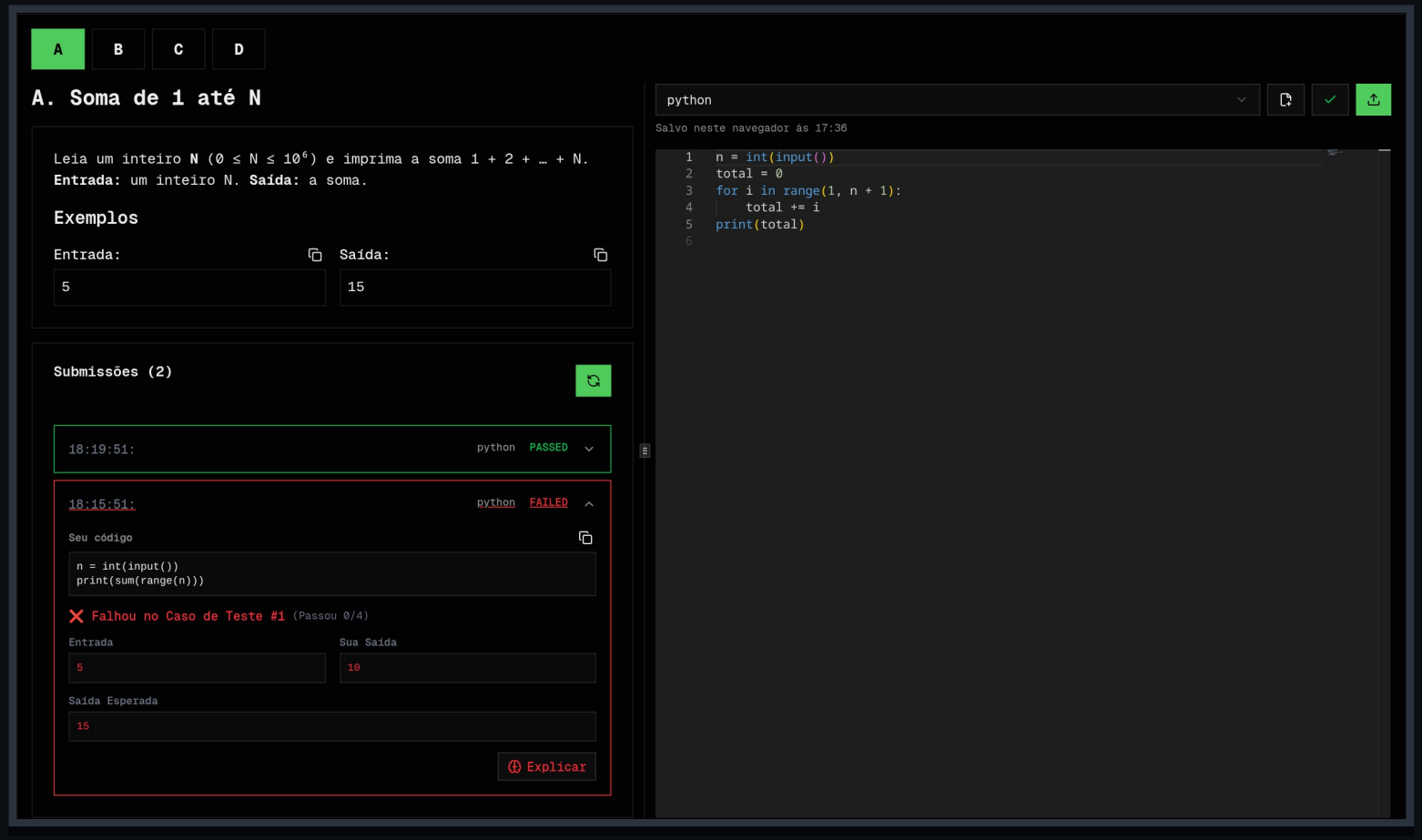
Dificuldade
fácil, médio ou difícil.

Tags
por tema, agrupadas: fundamentos, estruturas, algoritmos.

Meus
só os problemas que eu criei, inclusive rascunhos.

Importar JSON
vários problemas de uma vez, com o formato documentado.

O aluno vê o que enviou e por que falhou



Torneio: enunciado, envios e editor lado a lado

Editor com destaque
O mesmo do VS Code (Monaco), em 6 linguagens.

Salvo automaticamente
No navegador, por problema e linguagem: “Salvo às 17:36”.

Cada envio guarda o código
Expande, compara com o veredito e copia com um clique.

Teste que falhou
Entrada, saída do aluno e saída esperada.

Explicar
Pede à IA uma dica sobre o erro, no limite do torneio.

O professor monta o torneio em uma tela



Configuração: datas, privado e nível de IA



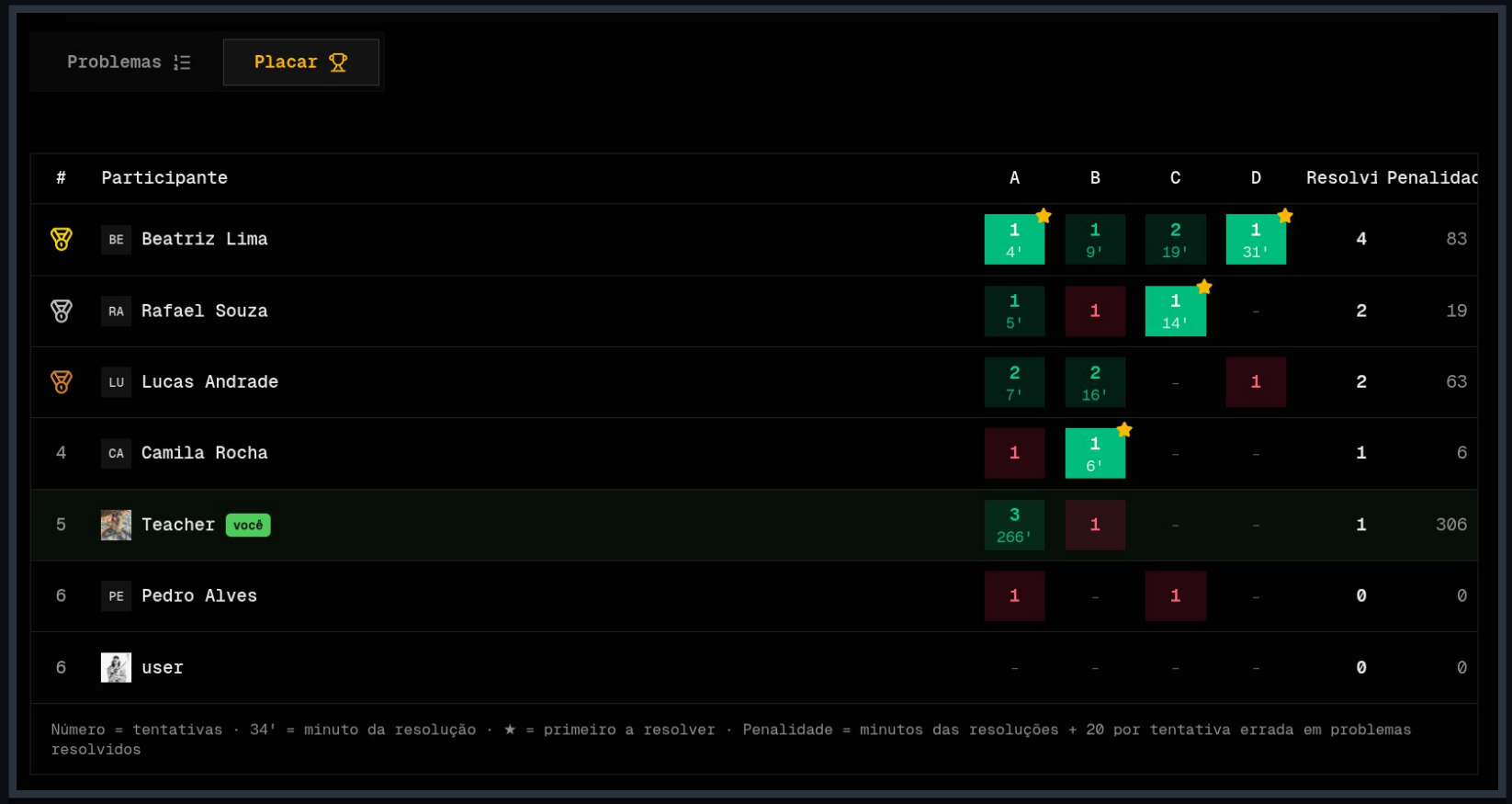
Problemas adicionados buscando pelo código

Torneio privado
participantes pedem para entrar e o professor aprova.

Assistência de IA
desligada, conceito, dica ou precisa; vale no próximo pedido.

Problemas
do banco, pelo código; viram A, B, C, D na ordem escolhida.

Placar no estilo das maratonas de programação



Placar de um torneio com dados de demonstração

1 4'
tentativas e minuto da resolução

★
primeiro a resolver o problema

verde / vermelho
resolvido / só tentativas erradas

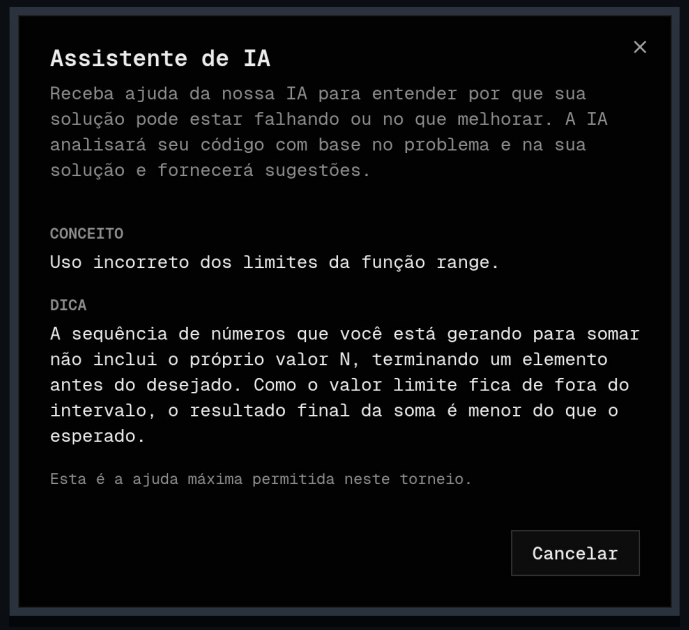
Penalidade
minutos das resoluções + 20 por tentativa errada (estilo ICPC)

Ao vivo
atualiza sozinho durante o torneio

O professor decide quanto a IA ajuda



Configuração do torneio: Desligada → Conceito → Dica → Precisa

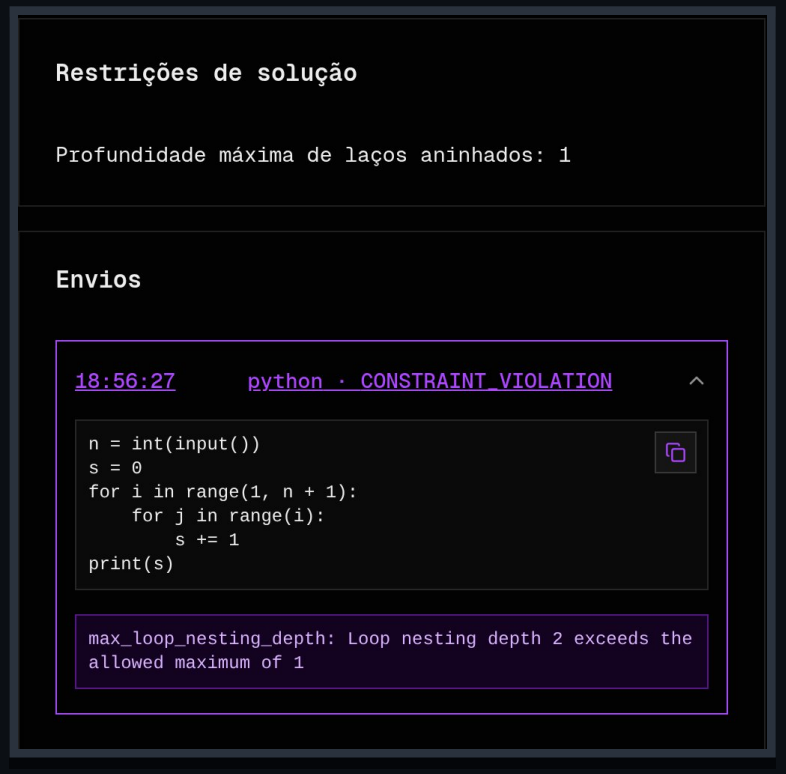


O aluno libera um nível por vez.
O servidor recusa qualquer pedido acima do limite do torneio, e uma mudança do professor vale já no próximo pedido.

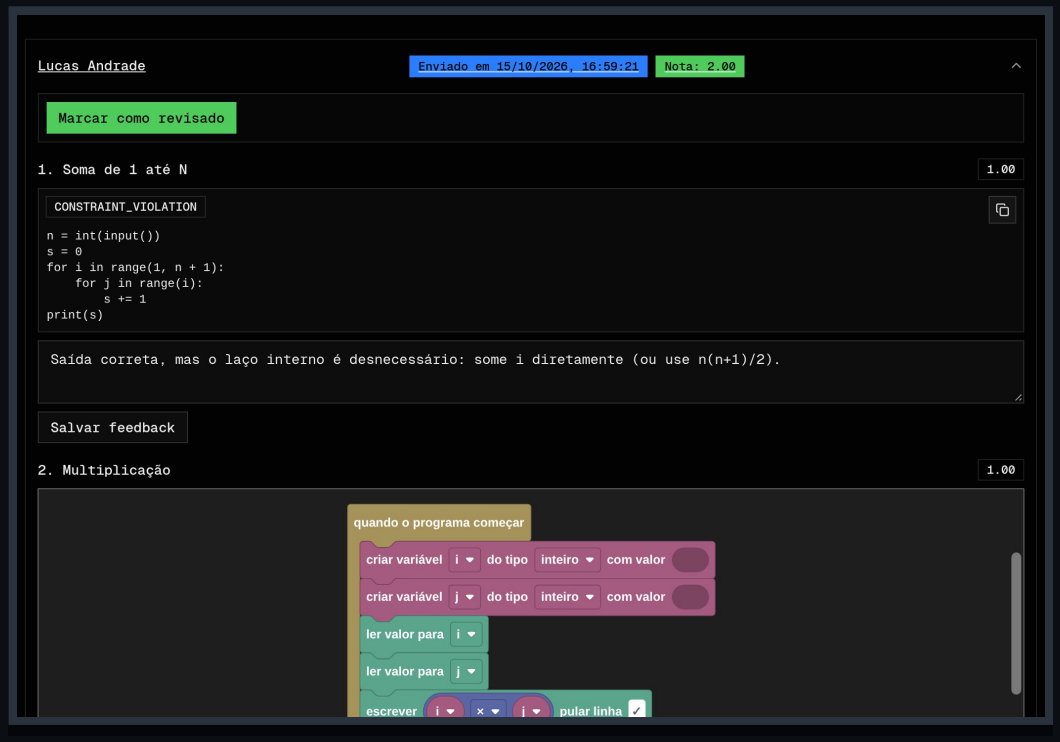
	IA DO WHYRUNNER	CHATBOT EXTERNO
Vê	código do aluno + teste que falhou	o que o aluno colar
Devolve	dica, sem a correção	solução pronta
Controle	professor, por torneio	nenhum
Limite	10 pedidos / 10 min	ilimitado
Formato	fala em blocos e linhas no modo visual	sempre código

O aluno ainda pode abrir outra IA. Por isso o professor também tem restrições de solução e atividades em sala.

O professor vê como o aluno resolveu, não só se acertou



Saída correta, mas viola "no máximo 1 laço"



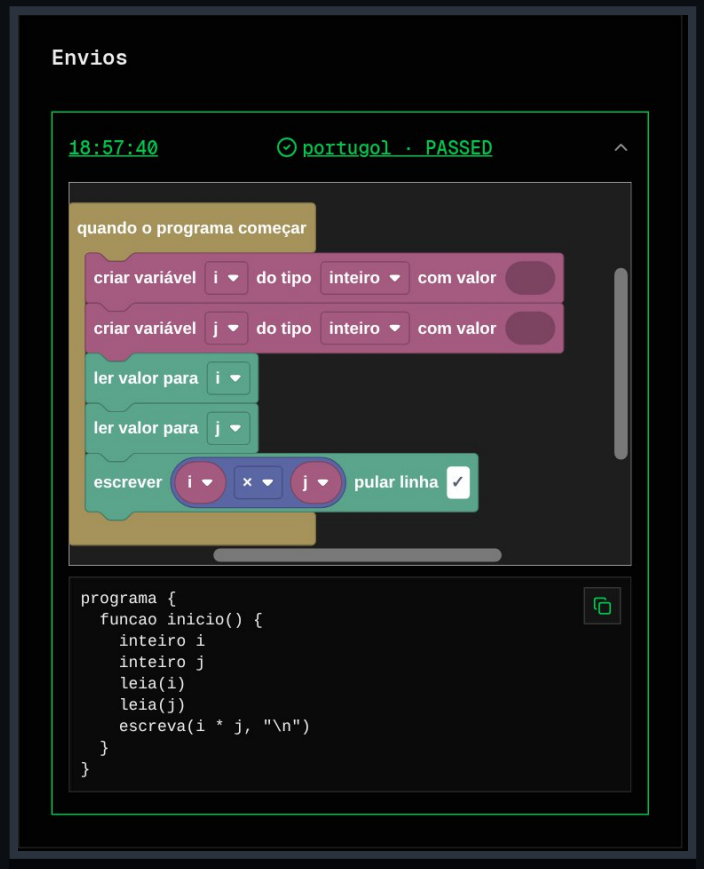
Revisão do professor: nota pelos testes + comentário; resposta em blocos

Restrições
limite de laços aninhados, construções proibidas ou "deve usar Dijkstra" (checado por IA)

Formatos
código, blocos (gera Portugol) ou Parsons (ordenar linhas)

Revisão
após o prazo, nota automática e comentário do professor

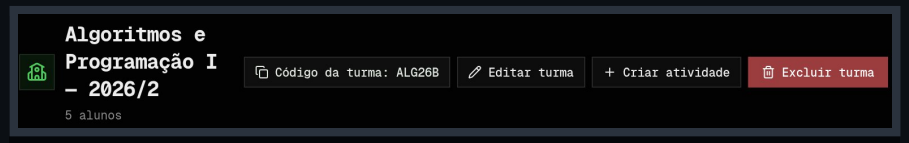
Blocos e Parsons para quem está começando



Blocos: a resposta vira Portugol e roda no judge



Parsons: ordenar e indentar linhas



Turma: código para os alunos entrarem

Blocos
Encaixe estilo Scratch. Bom para quem nunca programou.

Parsons
As linhas certas, fora de ordem. Treina leitura de código.

Código da turma
Seis letras, como ALG26B. O aluno digita e entra.

Aulas com prazo
O professor cria atividades; a revisão abre após o prazo.

Um teste informal já mudou o sistema

INFORMAL — 5 colegas usaram o sistema no meu servidor do Discord, sem roteiro nem questionário

O QUE INCOMODOU → O QUE MUDOU

Não dava para ver o código de cada tentativa no torneio
→ **Cada envio mostra o código, com botão de copiar**

O placar só mostrava medalhas
→ **Tentativas, minuto, ★ primeiro a resolver, penalidade, ao vivo**

Importar JSON dava “Invalid format: Required”
→ **Formato documentado e erro com o campo e o problema**

O QUE AGRADOU

Visual
O tema escuro, estilo terminal.

O nome
“Why” lido como o “Uai” mineiro: a ideia é mostrar por que o código falhou.

Não substitui a avaliação com a turma: amostra pequena, de colegas, sem professor.

O que falta: a avaliação em sala de aula

PLANEJADO

Aplicação

Um torneio e uma aula com uma turma real de programação.

PLANEJADO

Usabilidade

Questionário SUS com os alunos (nota de 0 a 100).

PLANEJADO

Visão do professor

Como foi criar a aula e o torneio, revisar envios e dosar a IA.

PLANEJADO

Uso

Análise descritiva: veredictos, tempo até acertar e pedidos de dica.

Ainda não aplicada. Sem ela, a pergunta de pesquisa fica sem resposta: o teste no Discord não mede aprendizado nem a experiência do professor.

O sistema está pronto; falta provar que ensina

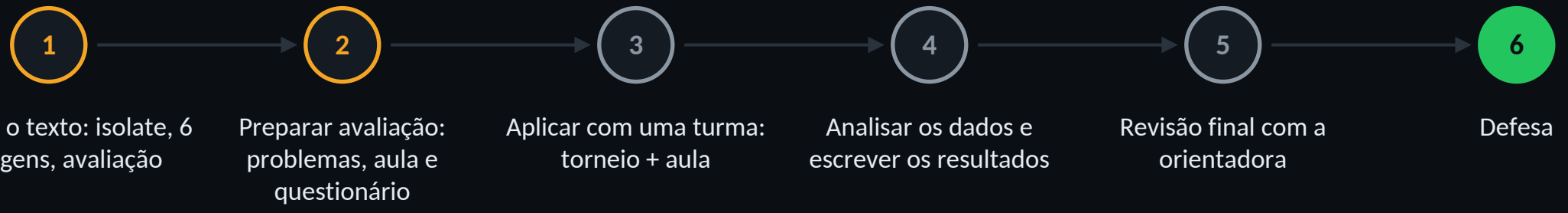
PONTOS FORTES

- \+ Sem broker de mensagens: só PostgreSQL
- \+ Isolamento no nível da IOI (MAREŠ; BLACKHAM, 2012), sem Docker privilegiado
- \+ Tipagem de ponta a ponta (Drizzle + Zod)
- \+ IA sob demanda, com entrada delimitada contra prompt injection

LIMITAÇÕES

- ! Ainda sem avaliação com alunos
- ! Não impede consulta a IAs externas
- ! Análise de laços por texto, não por AST: falso positivo em strings
- ! Depende da API de IA (custo e limites) e de Linux com cgroups

Próximas etapas, em ordem



Etapa crítica: aplicar com a turma (3). Sem ela não há resposta para a pergunta de pesquisa; as etapas 4 e 5 dependem dela.

Referências

ANTHROPIC. Claude Haiku 5.5. 2026. Disponível em: https://www.anthropic.com/claude.

BROOKE, J. SUS: a quick and dirty usability scale. Usability Evaluation in Industry, 1996.

CAMPOS, C. P.; FERREIRA, C. E. BOCA: um sistema de apoio a competições de programação. WEI, 2004.

CARNEIRO, M. F. Sistema web para auxílio no treinamento para maratonas de programação. UTFPR, 2012.

DRIZZLE TEAM. Drizzle ORM: TypeScript ORM for SQL. 2024.

GEMINI TEAM, GOOGLE. Gemini: a family of highly capable multimodal models. 2023.

LIMA, G. M. Gamma Online Judge: plataforma para as questões das maratonas UnB de programação. 2022.

MAREŠ, M.; BLACKHAM, B. A new contest sandbox. Olympiads in Informatics, v. 6, 2012.

MATSAKIS, N. D.; KLOCK, F. S. The Rust language. ACM SIGAda Ada Letters, 2014.

MERKEL, D. Docker: lightweight Linux containers for consistent development and deployment. Linux Journal, 2014.

RUN CODES. Run Codes: plataforma de programação online. 2024.

SILVA, J. T. S. et al. Ataux: uma ferramenta para auxiliar o acompanhamento de submissões de questões de programação. 2021.

STONEBRAKER, M.; ROWE, L. A. The design of POSTGRES. ACM SIGMOD, 1986.

VERCEL INC. Next.js: the React framework for the web. 2024.